Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No:2601004
Name: Samiksha M. Almast

Ass1: Create a Singly Linear Linked List with following operations

1. **Create Linked List**
2. **Traverse and print the node values**
3. **Insert node at a specific position**
4. **Find Middle node and print its value**
5. **Delete node**
6. **Reverse list**
7. **Calculate the sum of every two consecutive node values.**

Code:

```
# Singly Linear Linked List

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    # Create linked list by appending nodes
    def append(self, new_node):
        if self.head == None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next!= None:
                temp = temp.next
            temp.next = new_node
```

```
# Traverse and print nodes
def print(self):
    temp = self.head
    while temp!=None:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")

# Insert node at a specific position
def insert(self, new_node, pos):
    if pos < 1:
        print("Invalid position")
        return

    if pos == 1:
        new_node.next = self.head
        self.head = new_node
    else:
        temp = self.head
        p = 1

        while p < pos - 1 and temp!=None:
            temp = temp.next
            p += 1

        if temp==None:
            print("Invalid position")
            return

        new_node.next = temp.next
        temp.next = new_node

# Find middle node
def middle(self):
    middle = self.head
    last = self.head

    if self.head==None:
        print("List is empty")
        return

    while last!=None and last.next!=None:
```

```
                middle = middle.next
            last = last.next.next

        print("Middle node:", middle.data)

    # Delete node by value
    def delete(self, val):
        temp = self.head

        if temp!=None and temp.data == val:
            self.head = temp.next
            return

        while temp!=None and temp.next!=None:
            if temp.next.data == val:
                temp.next = temp.next.next
                return
            temp = temp.next

        print("Value not found")

    # Reverse linked list
    def reverse(self):
        prev = None
        temp = self.head

        while temp!=None:
            next_node = temp.next
            temp.next = prev
            prev = temp
            temp = next_node

        self.head = prev

    # Sum of every two consecutive nodes
    def pair_sum(self):
        temp = self.head

        while temp!=None and temp.next!=None:
            total = temp.data + temp.next.data
            print(temp.data, "+", temp.next.data, "=", total)
            temp = temp.next
```

```
list = LinkedList()
list.append(Node(10))
list.append(Node(20))
list.append(Node(30))
list.append(Node(40))
list.append(Node(50))
print("Original linked list:")
list.print()
print("\nInsert 100 at position 4:")
list.insert(Node(100), 4)
list.print()
print("\nMiddle node:")
list.middle()
print("\nDelete node 30:")
list.delete(30)
list.print()
print("\nReverse linked list:")
list.reverse()
list.print()
print("\nSum of consecutive nodes:")
list.pair_sum()
```

Output:

```
Original linked list:
10 -> 20 -> 30 -> 40 -> 50 -> None

Insert 100 at position 4:
10 -> 20 -> 30 -> 100 -> 40 -> 50 -> None

Middle node:
Middle node: 100

Delete node 30:
10 -> 20 -> 100 -> 40 -> 50 -> None

Reverse linked list:
50 -> 40 -> 100 -> 20 -> 10 -> None

Sum of consecutive nodes:
50 + 40 = 90
40 + 100 = 140
100 + 20 = 120
20 + 10 = 30
```